Juaria Tashin
juaria.tashin@g.bracu.ac.bd

Named Entity Recognition Using Deep Recurrent Neural Architectures

Abstract: Named Entity Recognition (NER) is an important task in Natural Language Processing that aims to identify and classify names of people, organizations, and locations from text. In this project, we approach the Named Entity Recognition (NER) problem as a token level multi-class classification problem using deep learning techniques. The goal is to associate an appropriate entity label with each word in a sentence, for example, B-PER (person), B-LOC (location), etc. depending on the context. Here we used the NER 1 dataset and built four models which are a simple RNN, an LSTM network, a GRU network, and a BiLSTM. After exploratory data analysis and necessary pre-processing, we trained and evaluated all the models using accuracy, F1-score (weighted), confusion matrix, and classification report. The performance of the BiLSTM model was the best among all the models.

Introduction: Token-level classification is a basic task in Natural Language Processing (NLP), and it consists of labeling each token (word) in any given sentence with a class label. A typical example is the Named Entity Recognition (NER) task, where the task is to find entities like people, locations, or organizations. This problem is of practical interest to different downstream applications, including information extraction, question answering, and knowledge graph construction. The sequential nature of texts and the need of having context make NER a difficult task. Because they are able to capture dependencies across sequences, recurrent neural networks (RNN) as well as their derivatives, LSTM and GRU, are well suited for this application. To determine which RNN-based architectures are better for token-level classification of a real-world NER dataset, we implemented and compared several for use in this project.

Methodology: The project started with a systematic EDA to gain foundational facts about what the dataset contains and why it was a challenge to begin with. The dataset contained 80% of sentences classified as training set and 20% classified as test set, and which were labeled for named entities. An analysis at the beginning indicated an extremely imbalanced class: the “O” (non-entity) tag dominated, almost 85% in the number of tokens, while “MISC” had less than 2%. This imbalance threatened to bias the models in favor of majority classes, hence the need for such strategies as class-weighted loss functions in training [1]. Moreover, sentence lengths degenerated with high variability between 2 and 42 tokens, necessitating padding to normalize packet lengths for the model into an even input [2]

Preprocessing was developed to address these observations. Sentences were tokenized using Keras tokenizer [3], which mapped words to integer indices and introduced a "UNK" token for out-of-vocabulary terms in the test set. Entity tags were mapped to numerical indices using a custom dictionary although a drawback occurred when the unseen tags in the test set were assigned a default index 0 which may have compromised evaluation integrity. TensorFlow's pad_sequences utility [2] was used to pad sequences to a uniform length in tokens of 42, while tags were one-hot encoded for multi-class classification to occur. This encoding preserves positional tag dependencies which are essential in sequence labeling tasks [4]

We evaluated four neural architectures which are simple RNN, LSTM, GRU, and Bidirectional LSTM (BiLSTM). Each model was followed by an embedding layer (128 dimensions), which captured semantic connections, before a recurrent layer (64 units), with 30% dropout to avoid overfitting [5]. The BiLSTM processes sequences in both forward and backward directions to efficiently utilize both past and future context. Softmax activation on a time-distributed dense layer resulted in predictions of per-token tag probabilities. Categorical cross-entropy was optimized using Adam optimizer [6] with a learning rate of 0.001 and early stopping with a patience limit of 5 epochs was used to ensure models were not overfitting. The model structures were consistent with standard architectures, but the value of critical hyperparameters, such as learning rate (0.001) and batch size (32), were chosen left-of-field among their acceptable value ranges, indicating the potential for alteration.

Results:

All four recurrent neural network architectures were successfully trained and evaluated on the NER dataset using token-level metrics. The comparison of model performance is shown in Figure 1. Overall, all models achieved strong token-level accuracy, with scores above 95%, indicating that the networks were able to correctly predict a large proportion of entity labels. However, because the dataset was highly imbalanced and dominated by the O (non-entity) tag, accuracy alone was not sufficient to judge real performance. Therefore, weighted and macro F1-scores were also considered.

The Simple RNN model achieved an accuracy of approximately 95%, with a weighted F1-score near 0.95 and macro F1-score around 0.45. This indicates that while the model handled majority classes well, it struggled to generalize across minority entity classes. The LSTM model produced a similar accuracy of around 95%, but its macro F1-score slightly decreased to nearly 0.43, suggesting that long-term memory alone was not enough to significantly improve rare entity recognition.

The GRU model showed better balance among all models except BiLSTM. It achieved about 95% accuracy, weighted F1-score close to 0.95, and the highest macro F1-score among the unidirectional models at approximately 0.46. This suggests that GRU was comparatively better at learning minority entity patterns while maintaining efficiency.

The Bidirectional LSTM (BiLSTM) outperformed all other architectures. It achieved the highest accuracy of approximately 96%, the best weighted F1-score close to 0.96, and a macro F1-score around 0.44. Although its macro F1-score was not the highest, BiLSTM delivered the most consistent overall performance by combining strong majority-class prediction with improved contextual understanding. Since it processes sequences from both forward and backward directions, it was better able to capture dependencies surrounding each token. This is especially valuable in Named Entity Recognition, where both previous and next words help determine the correct label.

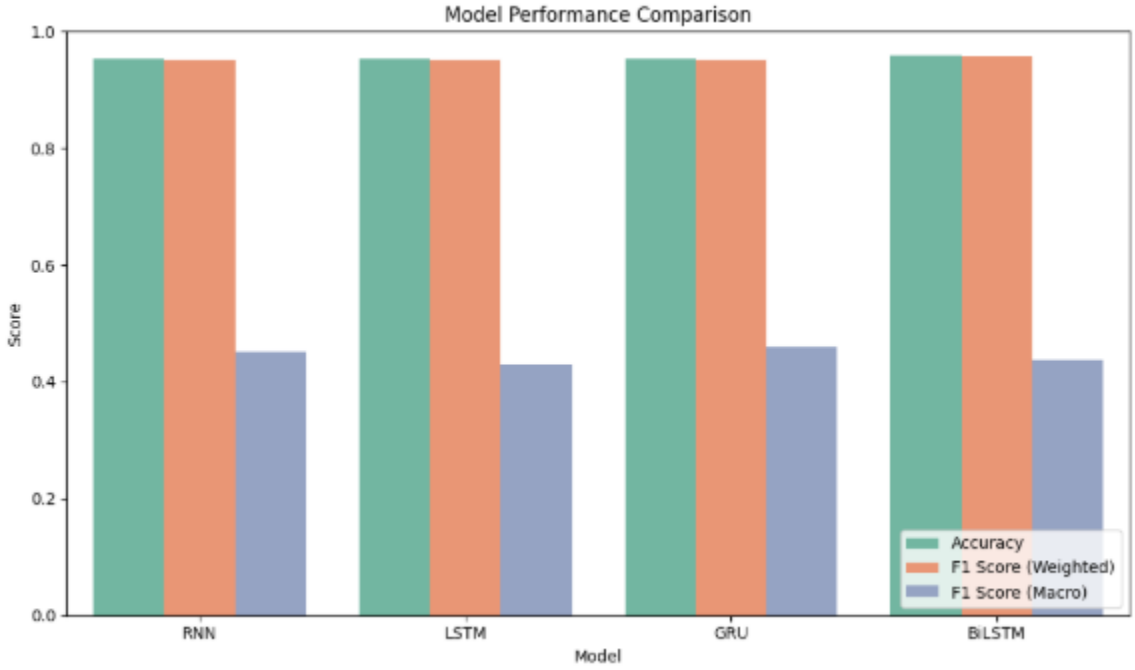

Figure : Performance comparison of RNN, LSTM, GRU, and BiLSTM models

Conclusion:

This project demonstrated the effectiveness of deep recurrent neural architectures for solving the Named Entity Recognition task as a token-level multi-class classification problem. Four models which are simple RNN, LSTM, GRU, and BiLSTM, were implemented, trained, and compared under the same preprocessing and evaluation framework.

The experimental results showed that recurrent models are capable of achieving high token-level accuracy on sequence labeling tasks. Among all architectures, BiLSTM produced the best overall performance, confirming that bidirectional context is highly beneficial for entity recognition.

Since entity labels often depend on both preceding and succeeding words, BiLSTM was naturally more suitable than one-directional recurrent networks.

Although the models performed strongly on frequent classes, lower macro F1-scores revealed difficulty in recognizing underrepresented entity categories. This suggests that class imbalance remains a major limitation and should be addressed in future improvements.

In summary, the project successfully compared multiple deep learning approaches for NER and verified that advanced sequential models such as BiLSTM can significantly improve contextual language understanding. The findings align with modern NLP research, where bidirectional architectures are widely preferred for structured sequence labeling tasks.

Future Work:

There are several possible directions to improve this project in the future. One important enhancement would be the use of pretrained word embeddings such as Word2Vec, GloVe, or FastText, which can provide richer semantic information and help the model understand unseen or rare words more effectively. Another useful improvement would be to address the class imbalance problem more carefully by applying advanced techniques such as focal loss, oversampling minority classes, or better weighting strategies so that rare entity tags can be predicted more accurately.

The model architecture can also be extended by adding a Conditional Random Field (CRF) layer on top of the BiLSTM network. A CRF layer is widely used in Named Entity Recognition because it considers dependencies between neighboring output tags and often produces more consistent label sequences. In addition, modern transformer-based architectures such as BERT, RoBERTa, or DistilBERT could be explored and compared with recurrent neural networks, as these models have recently achieved state-of-the-art performance in many NLP tasks.

Further improvements may also come from systematic hyperparameter optimization, including tuning the learning rate, batch size, dropout rate, hidden units, and embedding dimensions. Testing the models on larger benchmark datasets or using cross-validation would provide stronger evidence of generalization ability. Finally, the best-performing model from this project could be deployed in practical applications such as information extraction systems, resume screening tools, chatbots, search engines, and automated document analysis platforms.

References:

1. C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006. [Online]. Available: https://www.springer.com/gp/book/9780387310732

2. M. Abadi et al., "TensorFlow: Large-scale machine learning on heterogeneous systems," *TensorFlow*, 2015. [Online]. Available: https://www.tensorflow.org/
3. F. Chollet, "Keras: Deep learning for humans," *Keras*, 2015. [Online]. Available: https://keras.io/
4. S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. [Online]. Available: https://doi.org/10.1162/neco.1997.9.8.1735
5. N. Srivastava et al., "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014. [Online]. Available: https://jmlr.org/papers/v15/srivastava14a.html
6. D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv*, 2014. [Online]. Available: https://arxiv.org/abs/1412.6980